

Pintos Project 4: Virtual Memory

담당 교수 : 김영재

이름 / 학번 : 윤준서 / 20211558

개발 기간 : 25.12.1 - 25.12.11

I. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술

1. Page Table & Page Fault Handler

virtual page를 만들고 virtual memory를 효율적으로 다루기 위해 page table 관리를 개선한다.

2. Disk Swap

virtual memory를 disk 공간을 사용하여 확장할 수 있는 swap 기능을 설계 및 구현한다.

3. Stack Growth

프로그램이 필요로 하는 만큼 stack이 동적으로 확장할 수 있도록 한다.

II. 개발 범위 및 내용

A. 개발 범위

- 아래 각 항목 개발의 필요성 또는 개발 시 기대되는 결과를 간략히 서술

1. Page Table & Page Fault Handler

유효한 주소의 경우 virtual page를 찾아 physical page에 할당 후 mapping한다. 유효하지 않을 경우 stack을 확장한다. 그리고 page fault handler를 통해 프로그램 종료 시 할당되었던 자원을 해제한다. 이를 통해 시스템을 더욱 안정화하고 메모리를 절약할 수 있다.

2. Disk Swap

physical memory가 가득 찰 경우 disk swap 기능을 사용한다. LRU를 통해 eviction page를 정한다. 이를 통해 physical memory보다 큰 virtual memory를 사용 및 mapping할 수 있다.

3. Stack Growth

default limit을 기준으로 stack growth를 구현한다. 이를 통해 많은 메모리를 필요로 하는 프로그램을 실행할 수 있다.

B. 개발 내용

- 아래 항목의 내용만 서술

1. Page fault가 발생하는 이유와 이를 handling하는 전반적인 과정을 서술

physical memory에 없는 page에 접근을 하면 page fault가 발생한다. 이때 page table 내에서 유효한 주소일 경우, virtual page에서 파일을 load할 때 빈 frame을 할당한다. 이후 physical address와 virtual address를 mapping한다.

2. Disk swap 발생 시 사용한 page replacement algorithm에 대해 서술

frame을 할당할 때, 남은 frame이 없으면 page를 대체해야 한다. 이때 second chance algorithm을 사용한다. 이는 FIFO queue를 사용하고 LRU와 유사한 기법이다.

3. Stack growth 구현 시 stack 확장 여부를 판단할 수 있는 방법에 대해 서술

fault address가 user 영역 또는 kernel 영역에 있는지 확인한다. 그리고 default limit을 넘어가는지 확인한다. 마지막으로 fault address가 stack point와 4B 이내 혹은 32B 아래에 있는지 확인한다. 모든 확인이 완료되면 stack을 추가 할당한다.

III. 추진 일정 및 개발 방법

A. 추진 일정

- II. A. 개발 범위를 포함하여 구현 내용에 대한 일정 작성

12.1 - 12.4 : Page Table & Page Fault Handler

12.5 - 12.8 : Disk Swap

12.9 - 12.10 : Stack Growth

12.11 : Report

B. 개발 방법

- II. B.의 개발 내용을 구현하기 위해 각각에 대해 다음 사항들을 포함하여 설명

- 수정해야하는 소스코드

page table 구현 및 관리와 이와 연관된 virtual memory를 다루는데 필요한 API를 관리하는 vm 파일들을 새로 생성한다. 명세서를 따라 page.h, frame.h, swap.h와 해당 C 파일들을 추가했다. 또한 page-merge-mm test case를 해결하는 데 필요한 memory mapping 기능을 다루는 mmap.h와 해당 C 파일을 추가했다.

init.c에서 frame lock, swap lock을 init하는 기능을 추가한다.

syscall.c에서 memory mapping과 관련된 system call인 MMAP와 MUNMAP를 다루는 부분을 추가한다.

- 수정하거나 추가해야 하는 자료구조

struct thread에 page table과 memory list, index를 추가한다.

- 수정하거나 추가해야 하는 함수

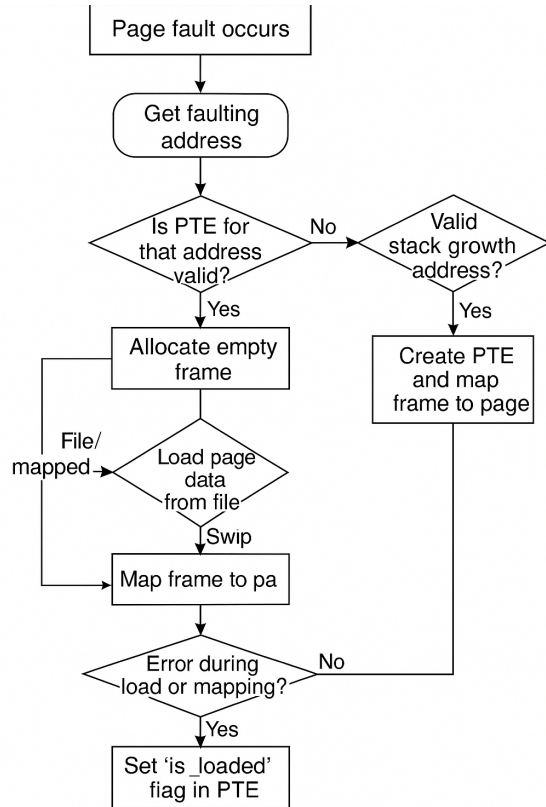
exception.c에서 page fault 함수에 stack의 확장 가능성 여부를 판단하는 기능을 추가한다.

process.c에서 page fault에 따른 stack growth의 기능을 추가한다.

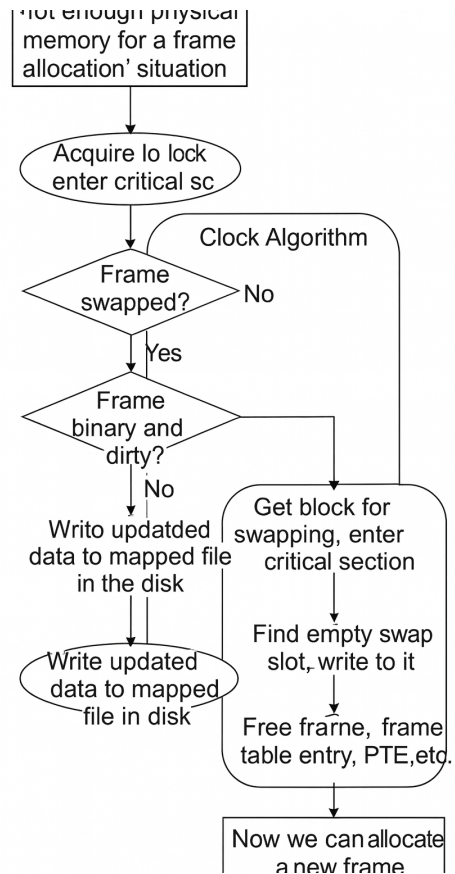
IV. 연구 결과

A. Flow Chart

- II. B. 개발 내용의 각 항목에 대하여 Flow Chart 작성



Page Table & Page Fault Handler & Stack Growth



Disk Swap

B. 제작 내용

- II. B. 개발 내용의 각 항목에 대하여 실질적으로 구현한 코드의 관점에서 작성 (구현 내용, 알고리즘 등을 명확히 서술할 것)
- ✓ 구현에 있어 Pintos에 내장된 라이브러리나 자체 제작한 함수를 사용한 경우 이에 대해서도 설명

thread.h - struct thread 내부

```
/* Virtual Memory */
struct hash pt;
struct list mmap_list;
int next_mapid;
```

thread 구조체에 virtual memory와 관련된 새로운 요소들을 추가했다.

struct hash pt는 page의 정보(file, swap, stack 등)를 저장하는 구조체다. lazy loading 시 해당 pt를 통해 data의 위치를 파악한다. 그리고 page가 eviction 되었을 때, 어느 swap slot에 저장되었는지를 확인할 수 있다. 프로세스가 종료될 때 해당 table을 순회하며 할당된 physical frame과 swap slot을 해제한다.

struct list mmap_list는 virtual memory에 mapping된 파일을 관리하는 구조체다.

int next_mapid는 fd처럼 mapping 영역을 구분하기 위한 식별자 정수다.

exception.c - page_fault() 내부

```
if (is_kernel_vaddr(fault_addr) || fault_addr == NULL) {
    sys_exit(-1);
}

if (!not_present) {
    sys_exit(-1);
}

struct pt_entry *pte = pt_find_entry (fault_addr);

if (pte != NULL) {
    if (!handle_mm_fault (pte)) {
        sys_exit (-1);
    }
}
else {
    if (!stack_growth (fault_addr, f->esp)) {
        sys_exit (-1);
    }
}
```

page fault 함수에 stack의 확장 가능성 여부를 판단하도록 구현했다.

첫째, kernel 주소거나 NULL pointer일 경우 즉시 종료한다.

둘째, Read Only 상태에서 Write를 시도한 것이라면 즉시 종료한다.

셋째, page table을 통해 유효한 주소인지 확인한다. 유효한 주소일 경우 해당 page를 load하고, 유효하지 않다면 stack을 growth한다.

process.c

```
bool stack_growth (void *addr, void *esp) {
    bool is_valid = is_user_vaddr (addr) &&
        (addr >= (void *) (PHYS_BASE - 0x8000000)) &&
        (addr >= (esp - 32));

    if (!is_valid) return false;

    void *upage = pg_round_down (addr);
    struct frame *kpage = vm_alloc_page (PAL_USER | PAL_ZERO);

    if (kpage == NULL) return false;

    /* 설치 먼저 시도 */
    if (!install_page (upage, kpage->kaddr, true)) {
        vm_free_page (kpage->kaddr);
        return false;
    }

    /* PTE 생성 */
    struct pt_entry *pte = allocate_and_init_pte(upage, SWAPPED, NULL, 0,
        0, 0, true);

    if (pte == NULL) {
        return false;
    }

    pte->is_loaded = true;
    kpage->pte = pte;

    /* Hash 등록 */
    return !pt_insert_entry (&(thread_current ())->pt, pte);
}
```

addr이 user 영역 주소이고, stack의 크기가 8MB 이내인지 확인한다. 그리고 esp와 stack의 거리가 32B 이내일 경우 유효하다고 판단한다.

이후 0으로 초기화된 page를 할당받는다. 그리고 이를 virtual address에 mapping한다.

이후 새로운 page table entry를 생성한다. 유형은 SWAPPED로 설정하고, 로드된 상태로 설정한다.

마지막으로 새로 생성한 pte를 hash table에 등록하여 관리한다.

```
bool handle_mm_fault (struct pt_entry *pte) {
    if (pte == NULL) return false;

    struct frame *frm = vm_alloc_page(PAL_USER);
    if (frm == NULL) return false;

    frm->pte = pte;
    bool load_result = false;

    switch (pte->type) {
        case BINARY:
            load_result = load_file_to_page(frm->kaddr, pte);
            break;

        case SWAPPED:
            swap_in(pte->swap_slot, frm->kaddr);
            load_result = true;
            break;

        default:
            load_result = false;
            break;
    }

    /* 로드 실패 시 정리 */
    if (!load_result) goto error_cleanup;

    /* 페이지 테이블 매핑 */
    if (!install_page(pte->vaddr, frm->kaddr, pte->writable)) {
        goto error_cleanup;
    }

    pte->is_loaded = true;
    return true;

error_cleanup:
    vm_free_page(frm->kaddr);
    free(frm);
    return false;
}
```

virtual memory에는 존재하지만, physical memory에 존재하지 않을 경우 호출된다. lazy loading 또는 swap out된 page를 다시 memory로 가져온다.

인자 page의 존재 여부를 확인하고, physical frame을 할당한다. 그리고 page의 유형에 따라 데이터를 어디서 가져올지 결정한다. BINARY일 경우, disk의 실행 파일에서 해당 offset data를 physical frame으로 읽는다. SWAPPED일 경우 swap disk에 저장되어 있던 data를 physical frame으로 복구한다.

이후 상태 업데이트를 하고 종료한다. 중간에 에러가 발생하면 physical memory, frame을 해제하여 memory waste를 방지한다.

다음 페이지부터는 **VM directory**에서 새로 생성한 파일들의 주요 부분들에 대한 내용이다.

frame.c

```
static void vm_second_chance (void) {
    // 초기화
    frame_clock = vm_frame_next ();
    lock_acquire (&frame_lock);

    if (!frame_clock) {
        lock_release (&frame_lock);
        return;
    }

    struct frame *victim = list_entry (frame_clock, struct frame, frame_elem);

    // Victim 선정
    while (true) {
        bool has_pte = (victim->pte != NULL);
        bool is_accessed = has_pte && pagedir_is_accessed (victim->thread->pagedir, victim->pte->vaddr);

        if (!is_accessed) break; // Accessed 비트가 0이면 무조건 탈출 (Victim 선정)

        // Accessed 비트 청소 (1 -> 0)
        pagedir_set_accessed (victim->thread->pagedir, victim->pte->vaddr, 0);

        // 다음 후보 탐색
        frame_clock = vm_frame_next ();
        if (!frame_clock) {
            lock_release (&frame_lock);
            return;
        }
        victim = list_entry (frame_clock, struct frame, frame_elem);
    }

    // 스왑 아웃 여부 결정
    bool need_swap = false;
    if (victim->pte != NULL) {
        bool is_binary_dirty = (victim->pte->type == BINARY) &&
                                pagedir_is_dirty (victim->thread->pagedir, victim->pte->vaddr);
        bool is_swapped_type = (victim->pte->type == SWAPPED);
        need_swap = is_binary_dirty || is_swapped_type;
    }

    // 스왑 및 정리 수행
    if (need_swap) {
        victim->pte->swap_slot = swap_out (victim->kaddr);
        if (victim->pte->type == BINARY) victim->pte->type = SWAPPED;
    }

    if (victim->pte) {
        victim->pte->is_loaded = false;
        pagedir_clear_page (victim->thread->pagedir, victim->pte->vaddr);
    }

    vm_delete_frame (victim);
    palloc_free_page (victim->kaddr);
    free (victim);
    lock_release (&frame_lock);
}
```

vm_second_chance() 함수는 Clock Algorithm을 구현하여 physical memory가 가득 찼을 때, **어떤 frame을 eviction할 지 결정하고 처리**한다.

첫째로, clock hand를 다음 frame으로 옮긴 후 lock을 건다.

다음, 해당 페이지가 최근에 참조되었는지 accessed bit를 확인한다. 0이라면, victim으로 선정하고 반복문을 종료한다.

선정한 page가 BINARY일 경우, 버린다. SWAPPED일 경우 swap out한다.

마지막으로 pte에 없음을 표시하고 pagedir_clear_page를 통해 더 이상 접근할 수 없도록 한다. 그리고 frame list에서 제거하고 physical memory를 반환한다.

```
bool load_file_to_page (void *kaddr, struct pt_entry *pte) {
    /* 현재 스레드가 이미 filesys_lock을 가지고 있는지 확인 */
    bool lock_held = lock_held_by_current_thread (&filesys_lock);

    /* 락을 안 가지고 있을 때만 획득 */
    if (!lock_held)
        lock_acquire (&filesys_lock);

    size_t bytes_read = file_read_at(pte->file, kaddr, pte->read_bytes, pte->offset);

    /* 락을 획득했던 경우에만 해제 */
    if (!lock_held)
        lock_release (&filesys_lock);

    if (bytes_read != pte->read_bytes) return false;

    if(pte->zero_bytes > 0) memset (kaddr + pte->read_bytes, 0, pte->zero_bytes);

    return true;
}
```

load_file_to_page() 함수는 **disk에 있는 data를 physical memory로 읽어온다**. BINARY 유형의 page를 lazy loading할 때 호출된다. pte에 저장된 file pointer, offset 을 이용해 정확한 위치에서 data를 읽어 kaddr(physical memory)에 쓴다.

page.h, page.c

```
struct pt_entry
{
    struct file *file;
    size_t offset;
    size_t read_bytes;
    size_t zero_bytes;

    /* 가상 주소 및 상태 정보 */
    void *vaddr;
    size_t swap_slot;

    /* 상태 플래그 */
    bool is_loaded;
    bool writable;
    pt_type type;

    struct hash_elem elem;
};
```

```
typedef enum {
    BINARY,
    ANON,
    SWAPPED
} pt_type;
```

page.h에서 정의한 **page table entry struct**와 **page의 type**을 정의한 enum이다. BINARY, ANON, SWAPPED로 분류했다.

pte 구조체에선 page의 data가 담긴 file pointer와 offset, 파일에서 실제로 읽어야 할 byte와 읽은 data 뒤에 채워 넣을 zero의 byte 수가 있다.

또한 virtual address, 저장된 swap disk의 slot 정보, 상태 플래그들을 정의했다.

마지막으로 hash table에 저장하기 위해 elem을 추가했다.

```
static unsigned pt_hash (const struct hash_elem *h_elem, void *aux UNUSED) {
    const struct pt_entry *p = hash_entry (h_elem, struct pt_entry, elem);
    return hash_int ((int) p->vaddr);
}

static bool pt_cmp (const struct hash_elem *left, const struct hash_elem *right, void *aux UNUSED) {
    const struct pt_entry *l = hash_entry (left, struct pt_entry, elem);
    const struct pt_entry *r = hash_entry (right, struct pt_entry, elem);

    return l->vaddr < r->vaddr;
}

static void pt_destroy_func (struct hash_elem *h_elem, void *aux UNUSED) {
    if (h_elem == NULL) return;

    struct pt_entry *ve = hash_entry (h_elem, struct pt_entry, elem);

    if (ve->is_loaded)
    {
        struct thread *cur = thread_current ();
        void *frame_addr = pagedir_get_page (cur->pagedir, ve->vaddr);

        if (frame_addr != NULL) vm_free_page (frame_addr);
    }

    free (ve);
}
```

다음은 pintos의 hash library를 이용하기 위해 page.c에서 정의한 helper 함수들이다.

pt_hash()는 hash table의 bucket index를 결정하기 위한 hash 값을 생성한다.

pt_cmp()는 hash table 내부에서 요소들을 정렬하거나 검색할 때 사용한다. 두 pte의 virtual address를 비교하여, 왼쪽이 작으면 true를 반환한다.

pt_destroy_func()는 hash table이 파괴될 때, table에 들어있던 각 entry를 정리하는 함수다. 프로그램이 종료될 때 호출된다.

swap.c

```
void swap_in (size_t index, void *kaddr) {
    if (index == 0) PANIC ("swap_in: invalid index 0");

    lock_acquire (&swap_lock);

    /* 실제 비트맵 인덱스로 변환 (index - 1) */
    size_t real_idx = index - 1;

    swap_block = block_get_role (BLOCK_SWAP);

    /* 비트맵 상태 업데이트 */
    bitmap_set_multiple (swap_bitmap, real_idx, 1, false);

    for (size_t i = 0; i < BLOCK_MAX; i++) {
        block_sector_t sec_no = (real_idx * BLOCK_MAX) + i;
        void *read_addr = kaddr + (i * BLOCK_SECTOR_SIZE);

        block_read (swap_block, sec_no, read_addr);
    }

    lock_release (&swap_lock);
}
```

swap_in() 함수는 swap disk에 저장된 data를 physical memory로 복구한다. page fault가 발생했을 때 호출된다.

bitmap을 0으로 설정하여 빈 공간으로 표시한다. 이를 통해 나중에 다른 page가 재사용할 수 있다. 그리고 block_read를 통해 sec_no에서 read_addr로 데이터를 복사(read)한다.

```
size_t swap_out (void *kaddr) {
    size_t slot_idx;

    lock_acquire (&swap_lock);

    /* 빈 슬롯 찾기 */
    slot_idx = bitmap_scan_and_flip (swap_bitmap, 0, 1, false);

    if (slot_idx == BITMAP_ERROR) PANIC ("swap_out: no swap slot available");

    swap_block = block_get_role (BLOCK_SWAP);

    size_t i = 0;
    block_sector_t start_sector = slot_idx * BLOCK_MAX; // BLOCK_MAX = 8

    while (i < BLOCK_MAX) {
        void *buffer_pos = kaddr + (i * BLOCK_SECTOR_SIZE);
        block_write (swap_block, start_sector + i, buffer_pos);
        i++;
    }

    lock_release (&swap_lock);

    return (slot_idx + 1);
}
```

swap_out() 함수는, physical memory에 있는 data를 swap disk의 빈 공간에 저장한다.

eviction이 발생할 때 호출된다. swap bitmap에서 연속된 1개의 빈 slot을 찾는다. 그리고 block_write를 통해 memory data를 512B씩 나누어 dist sector에 저장(write)한다.

mmap.h

```
struct mmap_file {
    int mapid;
    struct file *file;
    struct list_elem elem;
    void *vaddr;
    size_t size;
};
```

memory mapping 파일을 사용하기 위한 구조체를 정의했다.

mapping id, mapping된 파일 객체, thread의 mmap list를 연결하기 위한 elem, mapping 시작 virtual address, 마지막으로 mapping된 파일의 크기를 요소로 정의했다.

mmap.c

```
int mmap (int fd, void *addr) {
    struct thread *cur = thread_current ();

    if (addr == NULL || pg_ofs (addr) != 0) return -1;
    if (fd == 0 || fd == 1) return -1;
    if (fd < 2 || fd >= 128 || cur->fd[fd] == NULL) return -1;

    struct file *f = cur->fd[fd];

    lock_acquire(&filesys_lock);
    off_t file_size = file_length (f);
    lock_release(&filesys_lock);

    if (file_size == 0) return -1;

    for (off_t ofs = 0; ofs < file_size; ofs += PGSIZE) {
        if (pt_find_entry (addr + ofs) != NULL) return -1;
        if (!is_user_vaddr (addr + ofs)) return -1;
    }

    lock_acquire(&filesys_lock);
    struct file *reopened = file_reopen (f);
    lock_release(&filesys_lock);

    if (reopened == NULL) return -1;

    struct mmap_file *mf = malloc (sizeof (struct mmap_file));
    if (mf == NULL) {
        lock_acquire(&filesys_lock);
        file_close (reopened);
        lock_release(&filesys_lock);
        return -1;
    }

    mf->mapid = cur->next_mapid++;
    mf->file = reopened;
    mf->vaddr = addr;
    mf->size = file_size;

    size_t read_bytes = file_size;
    size_t offset = 0;
    void *upage = addr;

    while (read_bytes > 0) {
        size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes : PGSIZE;
        size_t page_zero_bytes = PGSIZE - page_read_bytes;

        struct pt_entry *pte = malloc (sizeof (struct pt_entry));
        if (pte == NULL) {
            free (mf);
            lock_acquire(&filesys_lock);
            file_close (reopened);
            lock_release(&filesys_lock);
            return -1;
        }

        pte->file = reopened; pte->offset = offset;
        pte->read_bytes = page_read_bytes;
        pte->zero_bytes = page_zero_bytes;
        pte->vaddr = upage; pte->is_loaded = false; pte->writable = true;
        pte->type = BINARY; pte->swap_slot = 0;

        if (!pt_insert_entry (&cur->pt, pte)) {
            free (pte); free (mf);
            lock_acquire(&filesys_lock);
            file_close (reopened);
            lock_release(&filesys_lock);
            return -1;
        }

        read_bytes -= page_read_bytes;
        offset += page_read_bytes;
        upage += PGSIZE;
    }

    list_push_back (&cur->mmap_list, &mf->elem);
    return mf->mapid;
}
```

mmap() 함수는 **file descriptor로 지정된 파일을 virtual address부터 시작하는 연속된 virtual page에 mapping**시킨다. 이때, 파일 data를 lazy loading으로 읽고, pte에만 생성한다.

첫째, 주소의 유효성 검사를 한다. 파일의 길이가 0인 경우 실패 처리한다.

둘째, 요청한 주소 범위에 이미 page가 존재하는지 확인한다. 존재할 경우 실패 처리한다.

셋째, file_reopen을 통해 해당 파일의 새로운 객체를 얻는다. 이를 통해 파일이 닫히거나 offset이 변경돼도 fd에 영향을 주지 않는다.

넷째, mapping 정보를 관리하기 위한 구조체를 할당하고, 고유 mapping id를 부여한다.

다섯째, 파일의 내용을 4KB 단위로 나누어 각 pte를 생성한다.

마지막으로, 모든 page를 등록하면 mmap_file 구조체를 mmap_list에 추가하고 mapid를 반환한다.

```

void munmap (int mapid) {
    struct thread *cur = thread_current ();
    struct list_elem *e;
    struct mmap_file *mf = NULL;

    for (e = list_begin (&cur->mmap_list);
         e != list_end (&cur->mmap_list);
         e = list_next (e)) {
        struct mmap_file *temp = list_entry (e, struct mmap_file, elem);
        if (temp->mapid == mapid) {
            mf = temp;
            break;
        }
    }

    if (mf == NULL) return;

    void *addr = mf->vaddr;
    size_t remaining_size = mf->size;

    while (remaining_size > 0) {
        struct pt_entry *pte = pt_find_entry(addr);

        if (pte != NULL) {
            if (pte->is_loaded) {
                if (pagedir_is_dirty (cur->pagedir, pte->vaddr)) {
                    bool need_lock = !lock_held_by_current_thread (&filesys_lock);
                    if (need_lock) lock_acquire (&filesys_lock);
                    file_write_at (mf->file, pte->vaddr, pte->read_bytes, pte->offset);
                    if (need_lock) lock_release (&filesys_lock);
                }
            }
            pt_delete_entry (&cur->pt, pte);
        }

        if (remaining_size >= PGSIZE) {
            addr += PGSIZE;
            remaining_size -= PGSIZE;
        }
        else remaining_size = 0;
    }

    list_remove (&mf->elem);
    file_close (mf->file);
    free (mf);
}

```

munmap() 함수는 mapid에 해당하는 mapping 파일을 해제한다. 이때, 변경된 data(dirty page)가 있다면 disk의 파일에 다시 기록(write back)한다.

첫째, 현재 thread의 mmap_list를 탐색하여 인자 mapid를 가진 파일을 찾는다.

둘째, mapping된 virtual address를 page 단위로 탐색한다. 각 주소에 해당하는 pte를 찾고, dirty check를 진행한다. 내용이 수정되었다면 변경 사항을 file_write_at을 통해 기록한다.

마지막으로, pt_delete_entry를 통해 pte를 제거하고 physical frame을 해제한다. 그리고 mmap_list에서 해당 구조체를 제거하고 memory를 해제한다.

- 개발 중 발생한 문제나 이슈가 있으면 이를 간략히 설명하고 해결한 방식에 대해 설명

처음 구현 후 make check를 했을 때 page-merge-mm이 fail을 했다. 이를 해결하기 위해 mmap.h, mmap.c 파일을 새로 만들고 system call을 추가해 memory map을 관리하도록 했다.

하지만 계속해서 lock aquire에서 panic이 발생했다. 이중 lock이 계속 발생했던 것인데, 원인을 찾는 데 오래 걸렸다. test의 out 파일의 로그와 filesystem lock, frame lock들의 위치를 디버깅했다.

page-merge-mm은 system call 중 MUNMAP을 통해 write를 실행하는 case로, system call 중 WRITE와 lock 사용이 겹친다. 이미 lock이 걸려 있는 경우 page fault를 호출하여 deadlock을 방지함으로써 test를 pass할 수 있었다.

C. 시험 및 평가 내용

- (채점 대상 테스트 케이스에 해당하는) make check 수행 결과를 캡처하여 첨부

```
pass tests/vm/pt-grow-stack
pass tests/vm/pt-grow-pusha
pass tests/vm/pt-grow-bad
pass tests/vm/pt-big-stk-obj
pass tests/vm/pt-bad-addr
pass tests/vm/pt-bad-read
pass tests/vm/pt-write-code
pass tests/vm/pt-write-code2
pass tests/vm/pt-grow-stk-sc
pass tests/vm/page-linear
pass tests/vm/page-parallel
pass tests/vm/page-merge-seq
pass tests/vm/page-merge-par
pass tests/vm/page-merge-stk
pass tests/vm/page-merge-mm
pass tests/vm/page-shuffle
```

모든 test case를 통과했다.